Enterprise RAG — Project Documentation

Repository: sai-teja33/enterprise-rag **Live demo:** enterprise-rag-beta.vercel.app

> This documentation was written by reading the actual source code of the repository (there is no README in the project), so it describes what the system does and how it is built, in plain language.

---

1. What is this project, in one paragraph?

Enterprise RAG is a **multi-tenant "chat with your company documents" system**. An organization (a "tenant") uploads its internal policy documents — leave policy, insurance policy, employee handbook, travel/expense policy, code of conduct, etc. — in PDF or DOCX format. The system reads those documents, breaks them into searchable pieces ("chunks"), turns those chunks into vector embeddings, and stores everything in a database. Employees (or an evaluator) can then ask natural-language questions like *"How many sick leaves am I allowed?"* and the system searches the tenant's own documents, finds the most relevant passages, and asks an LLM to produce an answer **grounded strictly in those documents**, complete with citations back to the source chunk, page, and file. If the documents don't actually contain the answer, the system is designed to say so instead of guessing.

In short: it's a **Retrieval-Augmented Generation (RAG) backend + admin frontend**, purpose-built for internal HR/policy Q&A, with tenant isolation, hybrid search, and answer-grounding safeguards baked in.

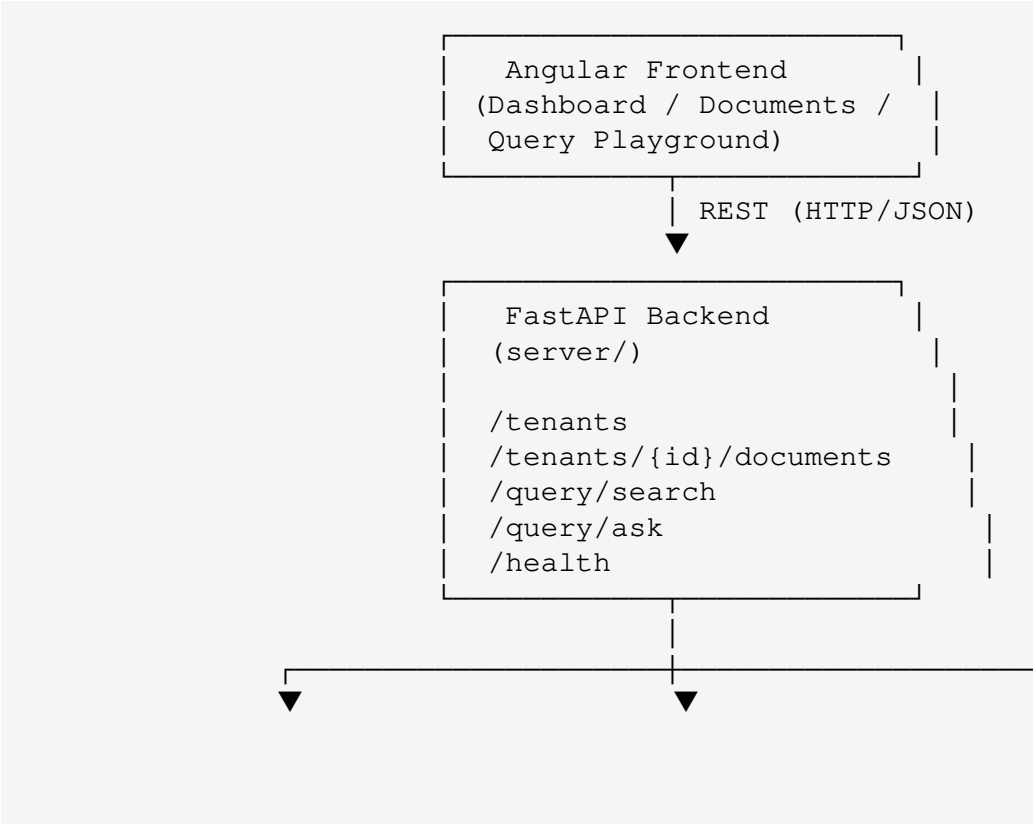
---

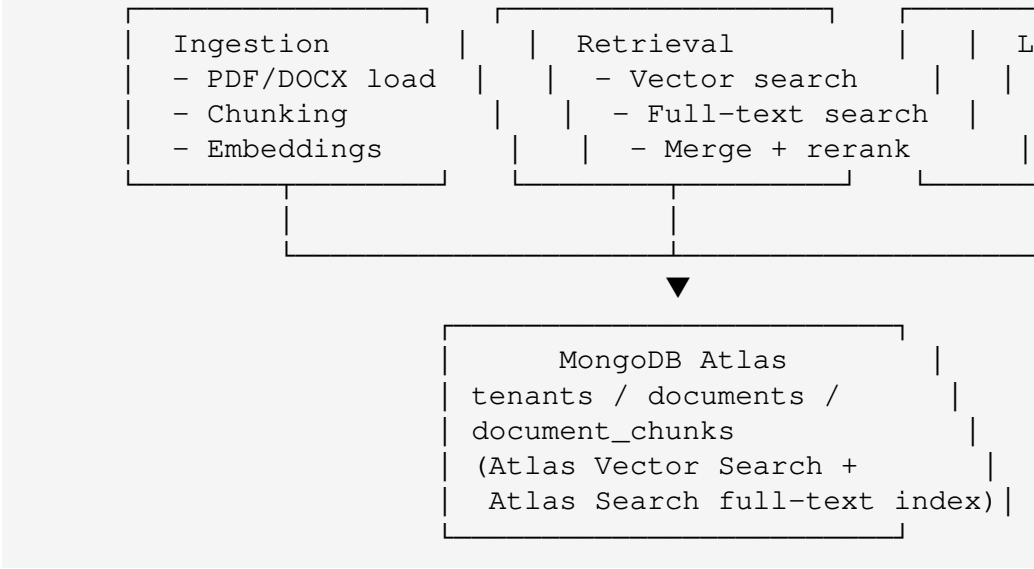
2. Who is it for?

- **HR** / **IT teams** at a company who want employees to be able to ask questions about internal policies instead of digging through PDFs.
- **Multiple organizations at once** — the system is multi-tenant, so several separate companies' document sets can live in the same deployment without ever mixing data.
- Anyone evaluating or learning how to build a **production-style RAG pipeline** (chunking strategy selection, hybrid retrieval, grounding/abstention logic, evaluation harness) — the codebase is also a good reference implementation.

---

3. High-level architecture





There is also a standalone **evaluation harness** (`eval/` + `scripts/run_eval.py`) that fires a fixed set of test questions at the `/query/ask` endpoint and scores the answers.

---

4. The backend (`server/`)

Built with **FastAPI** (Python). Entry point: `server/main.py`, which wires up CORS (so the Angular frontend can talk to it) and registers four route groups: tenants, documents, query, and health.

4.1 Tenants — multi-tenancy

`api/routes/tenants.py`

- `POST /tenants` — create a new tenant (organization) with a `name` and a unique `slug` (e.g. `acme-tech`). The slug is used everywhere else as the tenant's public identifier.
- `GET /tenants` — list all tenants.

Every document, chunk, and query is scoped to a `tenant_id`, and every database read/search filters on it — this is how the system keeps different companies' data completely separate while sharing the same infrastructure.

4.2 Documents — ingestion pipeline

`api/routes/documents.py`, backed by `ingestion/`, `chunking/`, and `db/`.

A document goes through a multi-step lifecycle, each exposed as its own API call so it can be inspected or re-run independently:

1. **Upload** — `POST /tenants/{tenant_id}/documents/upload` Accepts a PDF or DOCX file plus a `title` and `doc_type` (e.g. `leave_policy`, `insurance_policy`, `travel_policy`, `code_of_conduct`, `employee_handbook`). The file is saved to `data/raw/{tenant_slug}/` and a document record is created in MongoDB.

2. **Preview** — `GET /tenants/{tenant_id}/documents/{document_id}/preview` Parses the file and returns the first 3,000 characters of extracted text, useful for sanity-checking that parsing worked before committing to chunking.

3. **Chunk** — `POST /tenants/{tenant_id}/documents/{document_id}/chunk` Splits the document into chunks using a **doc-type-aware chunking strategy** (see §4.3 below), deletes any previous chunks for that document, and saves the new ones.

4. **Embed** — `POST /tenants/{tenant_id}/documents/{document_id}/embed` Generates vector embeddings (OpenAI `text-embedding-3-small`) for every chunk that doesn't have one yet, and stores the vectors alongside the chunk.

5. **Reprocess** — `POST /tenants/{tenant_id}/documents/{document_id}/reprocess` Convenience endpoint that deletes old chunks and redoes chunking + embedding in one call (useful after changing chunking logic).

6. **Bulk ingest** — `POST /tenants/{tenant_id}/documents/bulk-ingest` Scans a whole folder (`data/raw/{tenant_slug}/`) and processes every PDF/DOCX in it automatically. It **infers the `doc_type` from the filename** (e.g. a file with "leave" in its name becomes `leave_policy`, "insurance"/"medical"/"health" becomes `insurance_policy`, etc.), then runs chunk + embed for each new or changed file. This is the fast path for seeding a tenant with many documents at once.

7. **Status** — `GET /tenants/{tenant_id}/documents/status` Returns, for every document, how many chunks exist and how many are embedded, plus a `ready_for_query` flag — a quick health check on whether a tenant's knowledge base is fully indexed.

4.3 Chunking strategy

`chunking/chunker.py` picks a strategy based on the document's declared type:

Doc type	Chunking strategy
`leave_policy`, `insurance_policy`, `travel_policy`, `code_of_conduct`	`section_recursive` — splits along document sections/headings, then recursively within sections
`employee_handbook`	`page_section_recursive` — splits by page **and** section, since handbooks are usually longer and span many pages
anything else / unspecified	`recursive` — a generic recursive text splitter over the whole document

This matters because policy documents tend to have a lot of internal structure (headings like "Sick Leave", "Casual Leave", "Eligibility"), and chunking along that structure keeps each chunk topically coherent, which materially improves retrieval quality.

4.4 Retrieval — hybrid search

`retrieval/hybrid_retriever.py` combines two independent searches and merges the results:

- **Dense (vector) retrieval** — `retrieval/dense_retriever.py` embeds the user's question and runs a MongoDB Atlas `$vectorSearch` against the `chunk_vector_index`, filtered to the requesting tenant only.
- **Sparse (keyword / full-text) retrieval** — `retrieval/text_retriever.py` runs a MongoDB Atlas `$search` (full-text) query against the `chunk_text_index` over the chunk text, title, and doc type, again filtered to the tenant.

The two result sets are merged and de-duplicated by chunk ID (`merge_hybrid_results`), tracking each chunk's vector score, text score, and rank from each source. This hybrid approach means the system can find relevant chunks both when the question uses very similar wording to the source text (keyword search wins) and when it's phrased quite differently but semantically related (vector search wins).

A **reranking step** follows (`retrieval/reranker.py`). Interesting implementation detail: the code contains a fully-built cross-encoder reranker (`sentence-transformers/CrossEncoder`) that would re-score every candidate chunk against the question for more precise ordering — but it's currently **commented out and replaced with a pass-through** that just returns the top-k candidates as-is. The comment in the code explains this is to keep the free-tier deployment lightweight (cross-encoder models are relatively heavy to run). This is a good example of a real production trade-off between answer quality and hosting cost.

4.5 Answer generation and grounding

`llm/answer_generator.py` and `llm/prompts.py`.

The system supports two LLM providers, selected via config (`ANSWER_LLM_PROVIDER`):

- **OpenAI** (default, `gpt-4o-mini`)
- **Groq** (`llama-3.3-70b-versatile`), presumably as a faster/cheaper fallback

The system prompt (`SYSTEM_PROMPT`) instructs the model to act as an "enterprise policy assistant" that must answer **only** from the provided context chunks, never from outside knowledge, and to classify every answer into one of four modes:

- `direct_answer` — one clear, single answer is supported.
- `scoped_answer` — the policy varies by category/condition (e.g. different leave rules for different employee types), so the model must say so explicitly rather than flattening it into one answer.
- `partial_answer` — only part of the question is answerable from the retrieved chunks.
- `not_found` — the retrieved chunks don't contain reliable evidence; the model must return an exact fixed abstention string rather than inventing an answer.

The model returns structured JSON (answer text, answer mode, which chunk numbers it actually used, and reasoning notes about whether the case was multi-category or partially supported).

There's also a **"rescue pass"**: if the first LLM call abstains (`not_found`) but retrieval evidence was actually fairly strong (high vector similarity **and** either good lexical overlap or a reasonable rerank score), the system fires a second LLM call with a prompt nudging it to be less conservative and try harder to extract a partial/scoped answer from the same chunks — rather than giving up too early. This "rescue" only runs when the retrieval signal already looks solid, so it's a calibrated retry, not a way to force an answer out of weak evidence.

4.6 Guardrails before even calling the LLM

`retrieval/relevance_guard.py` computes a **lexical overlap score**: it strips stopwords from the question, checks what fraction of the meaningful question words actually appear somewhere in the retrieved chunks, and returns a `coverage_ratio`.

In `api/routes/query.py`'s `/query/ask` endpoint, before any LLM call happens:

- If the top vector similarity score is below `0.65` **and** lexical coverage is below `0.34`, the system abstains immediately (returns the fixed "could not find a reliable answer" message) without spending an LLM call.
- If lexical coverage is exactly `0`, it also abstains immediately.

This is a cost- and hallucination-control measure: cheap, fast heuristics filter out clearly-irrelevant queries before the (slower, costlier) LLM step, and reduce the chance of the model confabulating an answer from unrelated context.

4.7 API endpoints summary

Method	Path	Purpose
GET	`/`	Basic liveness message
GET	`/health`	API + MongoDB connectivity check
POST	`/tenants`	Create a tenant
GET	`/tenants`	List tenants
POST	`/tenants/{tenant_id}/documents/upload`	Upload a single PDF/DOCX
GET	`/tenants/{tenant_id}/documents`	List a tenant's documents
GET	`/tenants/{tenant_id}/documents/{document_id}/preview`	Preview extracted text
POST	`/tenants/{tenant_id}/documents/{document_id}/chunk`	Chunk a document

GET	`/tenants/{tenant_id}/documents/{document_id}/chunks`	List a document's chunks
POST	`/tenants/{tenant_id}/documents/{document_id}/embed`	Embed a document's chunks
POST	`/tenants/{tenant_id}/documents/{document_id}/reprocess`	Re-chunk + re-embed a document
POST	`/tenants/{tenant_id}/documents/bulk-ingest`	Auto-ingest a whole folder for a tenant
GET	`/tenants/{tenant_id}/documents/status`	Chunking/embedding status per document
POST	`/query/search`	Raw hybrid search — returns ranked chunks, no LLM
POST	`/query/ask`	Full RAG pipeline — retrieval + grounded LLM answer + citations

4.8 Data model (MongoDB collections)

- **`tenants`** — `name`, `slug`, `created_at`.
- **`documents`** — `tenant_id`, `title`, `doc_type`, `file_name`, `file_path`, `uploaded_at`, plus chunking metadata after processing.
- **`document_chunks`** — `tenant_id`, `document_id`, `title`, `doc_type`, `file_name`, `page_number`, `section_title`, `chunk_index`, `chunk_text`, `chunk_size`, `chunking_strategy`, and (once embedded) the `embedding` vector.

Two Atlas search indexes are used against `document_chunks`: `chunk_vector_index` (vector search over `embedding`) and `chunk_text_index` (full-text search over `chunk_text`, `title`,

`doc_type`).

---

5. The frontend (`frontend/client/`)

An **Angular 21** single-page app (using Angular Material and AG Grid for tabular data) with four main areas, matching the backend's capabilities:

- **Dashboard** — overview of tenants/documents status.
- **Documents** — upload documents, trigger chunking/embedding, view ingestion status per document.
- **Query Playground** — a UI for asking questions against a tenant's knowledge base and inspecting the answer, citations, and retrieval debug info returned by `/query/ask`.
- **Evaluation** — a UI surface presumably for viewing the results produced by the evaluation harness described below.

The app is structured with a `core/` layer (shared models and services, e.g. an HTTP client wrapper for the FastAPI backend) and a `features/` layer (one folder per screen), which is a standard, maintainable Angular project layout.

---

6. Evaluation harness (`eval/` + `scripts/run_eval.py`)

This is a self-contained **quality-testing tool** for the RAG pipeline, separate from the running app:

- `eval/eval_cases.json` contains dozens of hand-written test questions per tenant (e.g. `acme-tech`), each labeled with whether it *should* be answerable, which document types are expected to support the answer, and which keywords should appear.
- `scripts/run_eval.py` sends every test question to the live `/`

`query/ask` endpoint, compares the actual answer/citations against the expectations (normalizing raw `doc_type` values into broader categories like `leave_policy`, `insurance_policy`, `attendance_policy`, etc.), and writes out:
    - `eval/results/eval_results.json` — full per-question results
    - `eval/results/eval_results.csv` — the same, in spreadsheet form
    - `eval/results/eval_summary.json` — aggregate accuracy metrics

This lets the developer track whether changes to chunking, retrieval, or prompting improve or regress answer quality over a fixed, repeatable question set — essentially a regression test suite for the RAG system's correctness.

---

7. Configuration

Backend configuration (`server/core/config.py`) is loaded from environment variables / a `.env` file:

Variable	Purpose
`MONGODB_URI`, `DB_NAME`	MongoDB Atlas connection
`OPENAI_API_KEY` / `GROQ_API_KEY`	LLM + embedding provider credentials
`ANSWER_LLM_PROVIDER`	`"openai"` or `"groq"` — which LLM answers questions
`OPENAI_CHAT_MODEL` / `GROQ_CHAT_MODEL`	Which chat model to use per provider
`EMBEDDING_MODEL`, `EMBEDDING_BATCH_SIZE`	Embedding model settings
`RERANKER_MODEL`	Cross-encoder model name (currently unused — see §4.4)

`FRONTEND_ORIGINS`	Comma-separated list of allowed CORS origins for the Angular app

8. Tech stack summary

Layer	Technology
Backend framework	FastAPI (Python)
Database	MongoDB Atlas (documents + Atlas Vector Search + Atlas Search full-text index)
Embeddings	OpenAI `text-embedding-3-small` (via LangChain)
Answer LLM	OpenAI `gpt-4o-mini` or Groq `llama-3.3-70b-versatile`
Document parsing	`pypdf` (PDF), `docx2txt` (DOCX), via LangChain document loaders
Frontend	Angular 21, Angular Material, AG Grid
Evaluation	Custom Python harness driving the live API
Deployment	Vercel (frontend, per the "About" link on the repo)

9. Why the design choices matter (the interesting bits)

- **Doc-type-aware chunking** instead of one-size-fits-all splitting — chunk boundaries respect document structure so retrieved passages stay coherent.
- **Hybrid retrieval (vector + full-text)**, not just embeddings —

catches both semantically similar and lexically similar matches.
- **Two-layer abstention control**: cheap heuristic guardrails (vector score + lexical overlap) run *before* the LLM to cut cost and reduce hallucination risk, and the LLM itself is instructed to classify its own confidence (`direct_answer` / `scoped_answer` / `partial_answer` / `not_found`) rather than always giving a flat answer.
- **A calibrated "rescue pass"** that only retries when retrieval evidence was actually strong, avoiding both over-caution (abstaining when the docs did have the answer) and over-confidence (forcing an answer from weak evidence).
- **Strict tenant isolation** at the database query level, not just at the UI level — every retrieval call filters by `tenant_id`.
- **A real evaluation suite**, not just manual spot-checks — the project treats RAG quality as something to be measured and tracked over time.

---

10. Current limitations (as seen in the code)

- The cross-encoder reranker is implemented but disabled (pass-through only) to keep the free-tier deployment light — result ordering after the merge step is not re-ranked by relevance beyond the initial vector/text scores.
- Only PDF and DOCX are supported as document types.
- `doc_type` inference for bulk ingestion is filename-keyword-based, not content-based, so a mis-named file could be classified incorrectly.